■■ Project Blueprint: Garage (On-Demand Towing Ecosystem)

Author: Principal Software Architect, Google

Date: October 26, 2023

Status: Final Draft

Version: 1.0.0

■ Project Overview

Garage is an enterprise-grade, high-availability platform designed to bridge the gap between stranded motorists and towing service providers. In an era where "Everything-as-a-Service" is the norm, Garage digitizes the archaic roadside assistance industry. The system provides a real-time marketplace where users can request immediate towing assistance, track driver progress via GPS, and handle secure payments through a unified mobile and web interface.

The architecture is designed to handle high concurrency, ensuring that even during peak traffic hours or extreme weather conditions, the dispatching engine remains resilient and responsive.

■ Business Objective

* **Reduced MTTR (Mean Time to Rescue):** Optimize the dispatch algorithm to connect users with the nearest available driver.

* **Operational Transparency:** Provide real-time visibility into the towing lifecycle for both customers and service providers.

* **Scalable Monetization:** Implement a robust commission-based payment structure.

* **Trust & Safety:** Maintain a high standard of service through driver vetting and a bidirectional rating system.

■ User Roles

1. **Customer (Motorist):** Requests a tow, tracks the truck, and pays for the service.

2. **Tow Truck Driver:** Receives requests, navigates to the customer, and manages job status.

3. **Fleet Manager/Provider:** Manages a fleet of tow trucks, views earnings, and monitors driver performance.

4. **System Administrator (Google/Ops):** Manages the platform, handles disputes, and oversees system health.

■ Functional Requirements

* **Real-time Geolocation:** Live tracking of both the stranded vehicle and the incoming tow truck.

* **Smart Dispatching:** Automated matching engine based on distance, truck type (flatbed vs. hook), and driver availability.

* **Dynamic Pricing:** Estimation engine based on distance, time of day, and vehicle type.

* **Multi-Channel Notifications:** Push, SMS, and Email updates via Firebase Cloud Messaging (FCM).

* **Secure Payments:** Integration with Stripe/Braintree for PCI-compliant transactions.

* **Digital Proof of Service:** Photo uploads of the vehicle before and after the tow to prevent insurance fraud.

■ Non-Functional Requirements

* **Availability:** 99.99% uptime (Multi-region deployment).

* **Latency:** API response times < 100ms for core services; < 2s for map rendering.

* **Scalability:** Horizontal auto-scaling via Kubernetes to handle sudden spikes during weather events.

* **Security:** AES-256 encryption at rest; TLS 1.3 in transit. OAuth2/OpenID Connect for authentication.

* **Observability:** Integrated logging and tracing using Google Cloud Operations Suite (formerly Stackdriver).

■ Suggested Technology Stack

* **Frontend (Mobile):** Flutter (Single codebase for iOS/Android) for high-performance UI.

* **Frontend (Admin):** React.js with Tailwind CSS for internal dashboards.

* **Backend:** Go (Golang) for high-concurrency microservices.

* **Database (Transactional):** Google Cloud SQL (PostgreSQL) for relational integrity.

* **Database (Real-time):** Redis for caching and driver location indexing (Geo-hashing).

* **Messaging/Events:** Google Cloud Pub/Sub for asynchronous task processing.

* **Cloud Infrastructure:** Google Cloud Platform (GCP).

* **DevOps:** Docker, GKE (Google Kubernetes Engine), Terraform (IaC), GitHub Actions (CI/CD).

* **Authentication:** Firebase Auth or Google Identity Platform.

■ Suggested Folder Structure (Mono-repo Approach)

```text
/garage-platform
■■■ /services
■ ■■■ /auth-service # Identity & Access Management
■ ■■■ /dispatch-service # Matching algorithm & Job lifecycle
■ ■■■ /payment-service # Billing & Stripe integration
■ ■■■ /geo-service # Real-time GPS tracking (Redis-based)
■ ■■■ /notification-svc # FCM/SendGrid integration
■■■ /libs
■ ■■■ /shared-proto # gRPC / Protobuf definitions
■ ■■■ /common-go # Logging, tracing, and middleware
■■■ /apps
■ ■■■ /mobile-app # Flutter source code
■ ■■■ /admin-dashboard # React source code
■■■ /deploy
■ ■■■ /k8s # Kubernetes manifests
■ ■■■ /terraform # Infrastructure as Code
■■■ README.md
```

■ REST API Endpoints

Method	Endpoint	Purpose
`POST`	`/api/v1/requests`	Create a new tow request
`GET`	`/api/v1/requests/{id}`	Get status and driver details for a request
`PATCH`	`/api/v1/drivers/status`	Update driver availability (Online/Offline)
`GET`	`/api/v1/dispatch/nearby`	Find nearest available tow trucks (Admin/Internal)
`POST`	`/api/v1/payments/intent`	Initialize payment for the service
`PUT`	`/api/v1/jobs/{id}/complete`	Finalize tow and upload proof-of-delivery

■ Database Design (Core Tables)

`users`
* `id` (UUID, PK)
* `email` (String, Unique)
* `role` (Enum: CUSTOMER, DRIVER, ADMIN)
* `phone` (String)

`vehicles`
* `id` (UUID, PK)
* `user_id` (FK)
* `make_model` (String)
* `license_plate` (String)

`tow_requests`
* `id` (UUID, PK)
* `customer_id` (FK)
* `driver_id` (FK, Nullable)
* `pickup_location` (Geography/Point)
* `destination_location` (Geography/Point)
* `status` (Enum: PENDING, DISPATCHED, EN_ROUTE, COMPLETED, CANCELLED)
* `estimated_price` (Decimal)

`driver_locations` (Stored in Redis for performance)
* `driver_id` (Key)
* `lat_long` (Geo-spatial index)
* `last_updated` (Timestamp)

■ Deployment Architecture

The system utilizes a **Microservices Architecture** deployed on **Google Kubernetes Engine (GKE)**.

1. **Ingress:** Google Cloud Load Balancer (GCLB) provides global SSL termination.

2. **API Gateway:** Apigee or Kong for rate limiting and API orchestration.

3. **Compute:** GKE nodes running Go-based Docker containers.

4. **State:** Cloud SQL for persistent data; Cloud Memorandum (Redis) for real-time tracking.

5. **Analytics:** Data is streamed via BigQuery for long-term business intelligence and pattern recognition.

■ Development Roadmap

Week 1: Foundation & Architecture

* Infrastructure setup (GCP Project, Terraform scripts).

* Database schema finalization and migration scripts.

* CI/CD pipeline implementation.

Week 2: Authentication & User Management

* Identity provider integration (Firebase Auth).

* User/Driver profile microservices.

* Basic Mobile App UI skeleton (Flutter).

Week 3: Core Dispatch Logic

* Implementation of the matching algorithm.

* Geo-hashing integration using Redis for location tracking.

* Webhooks for status transitions.

Week 4: Real-time Features & Payments

* WebSocket/FCM integration for live updates.

* Stripe API integration for payments and payouts.

* Driver "Accept/Reject" flow.

Week 5: Admin Dashboard & Edge Cases

* Admin portal for managing disputes and fleet monitoring.

* Handling cancellations and refund logic.

* Integration testing for concurrent requests.

Week 6: Hardening & Launch

* Load testing (Simulating 10k+ concurrent users).

* Security audit and penetration testing.

* Final Beta release and Play Store/App Store submission.

Approved By:

Principal Software Architect, Google Cloud Platform Engineering